Selenium
From Zero to Viva-Ready

WebDriver, locators, waits, POM, TestNG, Grid, exceptions + 30 viva questions with answers — all in simple words

Quick-scan guide for SQA interviews and vivas

Companion to: API Testing Guide · Playwright Guide · JMeter Guide · CI/CD Guide · SQA A-Z Guide

PART 1 — What is Selenium?

Selenium is a free, open-source tool that **controls a real web browser with code** — it can open pages, click buttons, type into fields, and read what's on screen, exactly like a human, but automatically. You write test scripts (usually in Java or Python), and Selenium replays them in Chrome, Firefox, Edge, or Safari. It automates **web browsers only** — not desktop apps, not mobile apps (that's Appium).

The Selenium suite (viva favourite)

Component	What it is
Selenium WebDriver	The main tool — a library that drives the browser via code. When people say 'Selenium', they mean this.
Selenium IDE	A record-and-playback browser extension. Good for quick prototypes, not serious frameworks.
Selenium Grid	Runs tests on many machines/browsers in PARALLEL — one hub distributes tests to many nodes.

How WebDriver works (architecture)

Your script → WebDriver library → sends commands over HTTP using the **W3C WebDriver protocol** → a browser driver (chromedriver, geckodriver) → the real browser. The driver is the translator between your code and the browser. Since Selenium 4, this is a direct W3C standard (the old JSON Wire Protocol is gone) — a modern viva point.

PART 2 — Locators (how you find elements)

Before you click anything, you must FIND it. Selenium has 8 locator strategies:

Locator	Example	When
id	By.ID, 'login-btn'	Best choice — unique and fast
name	By.NAME, 'email'	Common on form fields
className	By.CLASS_NAME, 'btn-primary'	OK if class is unique
tagName	By.TAG_NAME, 'input'	Rarely alone — too generic
linkText	By.LINK_TEXT, 'Forgot password?'	Exact text of a link
partialLinkText	By.PARTIAL_LINK_TEXT, 'Forgot'	Part of link text
cssSelector	input[type='email']	Fast and powerful — preferred
xpath	//button[text()='Login']	Most flexible — can travel the DOM

XPath vs CSS (guaranteed viva question): CSS is faster and cleaner; XPath is more powerful — it can find elements by their TEXT and can walk UP to parent elements (CSS can't). Absolute XPath (/html/body/div[2]/...) breaks easily — always prefer relative XPath (//button[@id='save']).

PART 3 — Waits (the #1 viva topic)

Web pages load at unpredictable speeds. If your script clicks before the element exists, the test fails. Waits tell Selenium to be patient — and using them correctly separates juniors from seniors.

Wait	What it does	Verdict
Thread.sleep(5000)	Blindly freezes for 5 s no matter what	NEVER use — slow AND unreliable
Implicit wait	Global setting: poll up to X seconds every time any element is searched	Simple but one-size-fits-all
Explicit wait (WebDriverWait)	Wait up to X seconds for a SPECIFIC condition (visible, clickable, present)	Best practice — precise
Fluent wait	Explicit wait + custom polling interval + ignored exceptions	For tricky dynamic elements

```
wait = WebDriverWait(driver, 10)
element = wait.until(EC.element_to_be_clickable((By.ID, 'submit')))
```

Never mix implicit and explicit waits — the timeouts interact unpredictably. Pick explicit waits.

PART 4 — Everyday operations

Task	How
Open a page	driver.get('https://site.com')
Type text	element.send_keys('hello')
Click	element.click()
Read text	element.text
Dropdowns	Select(element).select_by_visible_text('Bangladesh')
Alerts (JS popups)	driver.switch_to.alert → .accept() / .dismiss() / .send_keys()
Frames (page inside a page)	driver.switch_to.frame('frameName') ... switch_to.default_content()
New windows/tabs	driver.switch_to.window(handle) using driver.window_handles
Scroll / JS	driver.execute_script('window.scrollBy(0,500)')
Screenshot	driver.save_screenshot('bug.png') — attach to bug reports
Hover / drag-drop	ActionChains(driver).move_to_element(el).perform()
Headless mode	options.add_argument('--headless=new') — browser runs invisibly (CI)

PART 5 — Framework concepts

Page Object Model (POM)

POM = one class per page. The class holds that page's locators and actions (LoginPage has username field, password field, and a login() method). Tests then read like English: loginPage.login('user','pass'). **Why it matters:** when the UI changes, you fix the locator in ONE place, not in 50 tests. Often paired with a Page Factory (@FindBy annotations in Java) to initialize elements.

Test runners: TestNG / JUnit / pytest

Selenium only drives the browser — it has NO assertions or test organization of its own. A test runner adds: @Test annotations, assertions, setup/teardown (@BeforeMethod/@AfterMethod), grouping, parallel execution, retry logic, and reports. TestNG extras worth naming: priorities, dependsOnMethods, @DataProvider for data-driven tests, testng.xml suites.

Data-driven testing

Same test, many data rows — from Excel (Apache POI), CSV, or TestNG @DataProvider. Example: one login test runs 10 times with 10 username/password combinations.

Common exceptions (memorize these 5)

Exception	Meaning / usual fix
NoSuchElementException	Locator wrong or element not loaded yet → fix locator / add explicit wait
StaleElementReferenceException	Page refreshed after you found the element → re-find it
TimeoutException	Explicit wait ran out → element never met the condition
ElementNotInteractableException	Element exists but hidden/disabled → wait for clickable
NoSuchWindowException / NoSuchFrameException	Switched to a window/frame that doesn't exist

PART 6 — 30 Viva Questions with Answers

Q: What is Selenium?

A: An open-source tool that automates web browsers via code — it clicks, types, and reads pages like a human. Web only; supports Java, Python, C#, JS across Chrome/Firefox/Edge/Safari.

Q: What are the Selenium components?

A: WebDriver (the main library), IDE (record-and-playback extension), and Grid (parallel execution on many machines). RC is dead/legacy.

Q: How does WebDriver talk to the browser?

A: Script → WebDriver library → W3C WebDriver protocol over HTTP → browser driver (chromedriver) → browser. Selenium 4 uses the W3C standard directly.

Q: What's new in Selenium 4?

A: Native W3C protocol, relative locators (above/below/near), better window/tab APIs, Chrome DevTools Protocol access, improved Grid.

Q: List the locator strategies.

A: id, name, className, tagName, linkText, partialLinkText, cssSelector, xpath — 8 total. Prefer id, then CSS.

Q: XPath vs CSS selector?

A: CSS is faster/cleaner; XPath can locate by text content and traverse to parents, which CSS cannot. Use relative XPath, never absolute.

Q: Absolute vs relative XPath?

A: Absolute starts from /html and breaks with any layout change. Relative (//) starts anywhere and is resilient. Always relative.

Q: How do you handle a dynamic element (changing id)?

A: Use stable attributes with contains()/starts-with() in XPath, CSS attribute selectors, anchor to nearby stable text, or ask developers for test ids.

Q: What are the wait types?

A: Implicit (global polling default), explicit/WebDriverWait (wait for a specific condition — best practice), fluent (explicit + custom polling and ignored exceptions). Thread.sleep is not a real wait.

Q: Why is Thread.sleep bad?

A: It always waits the full time (slow) and still fails if the element takes longer (unreliable). Explicit waits proceed the moment the condition is met.

Q: Implicit vs explicit wait?

A: Implicit applies to EVERY findElement globally; explicit targets ONE element with ONE condition like elementToBeClickable. Never mix them.

Q: findElement vs findElements?

A: findElement returns the first match or throws NoSuchElementException; findElements returns a list — empty list if none found, no exception.

Q: driver.get() vs driver.navigate().to()?

A: Both open a URL; navigate() additionally offers back(), forward(), refresh() history controls.

Q: close() vs quit()?

A: close() closes the current window only; quit() closes ALL windows and kills the WebDriver session. Always quit() in teardown.

Q: How do you handle dropdowns?

A: The Select class: select_by_visible_text / by_value / by_index. Works only on real tags — custom dropdowns need click sequences.

Q: How do you handle JS alerts?

A: driver.switch_to.alert, then accept(), dismiss(), text, or send_keys().

Q: How do you handle frames?

A: switch_to.frame(name/index/element) to enter, switch_to.default_content() to come back. Elements inside a frame are invisible until you switch.

Q: How do you handle multiple windows?

A: Save driver.window_handles, loop to the new handle, switch_to.window(handle), do work, switch back to the original handle.

Q: How do you take a screenshot?

A: driver.save_screenshot('name.png') — I attach screenshots to bug reports as evidence, and capture automatically on failure in teardown.

Q: What is Page Object Model and why use it?

A: One class per page holding its locators + actions. UI change = one-line fix instead of editing 50 tests. Improves readability, reuse, and maintenance.

Q: What is Page Factory?

A: A POM helper (@FindBy annotations + initElements) that initializes page elements lazily. Same goal as POM, more declarative syntax.

Q: What is TestNG and why do you need it?

A: Selenium has no test structure of its own. TestNG adds @Test, assertions, setup/teardown, grouping, priorities, @DataProvider, parallel runs, and HTML reports.

Q: Hard assert vs soft assert?

A: Hard assert stops the test at first failure. Soft assert records failures and continues, reporting all at the end (must call assertAll()).

Q: How do you run tests in parallel?

A: TestNG parallel attribute in testng.xml, and Selenium Grid to distribute across machines/browsers. Cuts a 2-hour suite to minutes.

Q: What is Selenium Grid?

A: Hub-and-node architecture: the hub receives tests and routes them to nodes with the requested browser/OS. Enables parallel, cross-browser execution.

Q: What is headless execution?

A: Browser runs without a visible window (--headless). Faster, and required on CI servers that have no display.

Q: What is StaleElementReferenceException and the fix?

A: You held a reference to an element, then the DOM refreshed — the reference died. Fix: re-locate the element after the refresh (or retry pattern).

Q: Can Selenium handle CAPTCHA or OTP?

A: No — by design. CAPTCHAs exist to block automation. Solutions: ask devs to disable in test environment, use test accounts, or bypass via API-issued session.

Q: How do you handle file upload?

A: If it's an : send_keys('C:/path/file.pdf') — no dialog needed. OS-level dialogs need tools like AutoIT/Robot (Selenium can't touch OS windows).

Q: Selenium vs Playwright/Cypress?

A: Selenium: most mature, all languages, real cross-browser standard, huge community — but needs manual waits and separate drivers. Playwright/Cypress: modern, auto-waiting, faster setup. I'd pick based on team stack; concepts transfer.

Last-minute quick scan

If they say...	You say...
Selenium?	Code-controlled real browser automation for web apps.
Components?	WebDriver + IDE + Grid.
Best locator?	id first, then CSS; XPath when you need text or parent traversal.
Waits?	Explicit WebDriverWait is best practice; never Thread.sleep; don't mix implicit+explicit.
close vs quit?	One window vs whole session.
POM?	One class per page → one-place fixes, readable tests.
Stale element?	DOM refreshed — re-find the element.
Grid?	Hub routes tests to nodes → parallel cross-browser runs.

Bridge line for viva: "My portfolio project is API-level automation (Postman/Newman/CI) — the same discipline applies at UI level with Selenium: locate, act, assert, and keep everything maintainable with POM."